5sosy: Strategic, Technical, and Product Architecture for Autonomous Multi-Agent Educational Orchestration

Introduction and Strategic Paradigm

The educational landscape in the Middle East and North Africa (MENA) region, particularly within the Arab Republic of Egypt, is currently defined by highly competitive, standardized academic pathways. The most critical of these is the *Thanaweya Amma* (secondary education) system, which dictates university admissions and subsequent socioeconomic mobility. Decades of systemic structural inefficiencies, including disproportionately high student-to-teacher ratios and underfunded public classrooms, have fostered a massive shadow education economy. This parallel system, reliant on private tutoring (locally known as *Deroos Khoseya*), places an immense financial burden on families. The private tutoring market in Egypt was valued at USD 0.78 billion in 2024 and is projected to reach USD 1.75 billion by 2033, expanding at a compound annual growth rate of 9.26 percent.[1] Market reliance is systemic; official estimates indicate that 81 percent of secondary school students, 64 percent of preparatory students, and 56 percent of primary students utilize private tutoring services to supplement their public education.[1]

The 5sosy project is conceived as an autonomous, multi-agent artificial intelligence study assistant designed specifically to democratize access to elite-level, curriculum-aligned tutoring. Submitted as a comprehensive blueprint for the Google for Startups AI Agents Challenge (Track 1: Build Net-New Agents), the platform shifts the computational paradigm away from static, hard-coded execution paths toward autonomous declarative intent.[1] The challenge mandates the construction of a reliable, production-ready system capable of driving real business results, with a strict submission deadline of June 5, 2026, at 5:00 PM PT (June 6, 2026, at 3:00 AM GMT+3).[1] The system targets the Ministry of Education's digital distribution channels, extracting and transforming dense, unstructured Arabic PDF textbooks into an interactive, multimodal cognitive mentor.[1]

To achieve enterprise-grade scalability and reliability by the submission deadline, the architecture relies exclusively on the Google Cloud ecosystem.[1] By leveraging the newly upgraded Agent Development Kit (ADK), the Model Context Protocol (MCP), and the Agent-to-Agent (A2A) protocol, the system orchestrates a decentralized network of specialized AI agents. This report details the exhaustive technical specifications, vector topology designs, multi-LLM comparative analyses, media synthesis pipelines, security guardrails, and market feasibility required to transition the 5sosy initiative from a local prototype located at C:\Users\hesh1\Desktop\my-agent into a production-ready enterprise

asset situated at C:\Users\hesh1\Desktop\5sosy.[1]

System Architecture and the Multi-Agent Blueprint

The transition from a monolithic chatbot architecture to a distributed, multi-agent framework is paramount for handling complex, multi-step reasoning tasks without degrading the context window of the underlying Large Language Model (LLM). The architectural blueprint for 5sosy utilizes a highly structured, graph-based topology governed natively by Google's Agent Development Kit (ADK).[1]

The Agent Development Kit (ADK) Infrastructure

The ADK provides a flexible, modular framework that applies rigorous software development principles to AI agent creation.[2] Rather than relying on simple iterative loops, the ADK supports sophisticated execution structures including LlmAgent constructs for primary reasoning, SequentialAgent constructs for linear workflows, ParallelAgent constructs for simultaneous task execution, and LoopAgent constructs for batch processing and continuous chronological updates.[1]

The core execution of the ADK is powered by an asynchronous Event Loop, which manages the yield, pause, and resume cycles of the agent network.[1] This architecture allows the 5sosy platform to support long-running tasks, such as generating an hour-long study plan or rendering a five-minute video, without blocking the primary user interface thread. The runtime environment supports graceful cancellation and the ability to resume an agent's execution from a previously saved state, a critical requirement for maintaining continuity in long-term educational mentorship.[1]

The Agent-to-Agent (A2A) Protocol Implementation

A defining architectural feature of the 5sosy platform is the exhaustive implementation of the Agent2Agent (A2A) protocol. Standardized within the ADK environment, A2A operates as an open communication standard designed to facilitate seamless interoperability within multi-agent systems.[3] It enables disparate AI agents—potentially built upon different frameworks such as LangChain or CrewAI—to discover, negotiate, and delegate tasks to one another independently of their underlying internal state or memory structures.[3]

Within the 5sosy ecosystem, the primary Orchestrator agent acts as the A2A Client, delegating specialized requests to remote A2A Servers representing the platform's sub-agents.[3] The system bypasses monolithic deployment vulnerabilities by wrapping individual sub-agents with the ADK's to_a2a() utility function.[5] This programmatic wrapper automatically complies an AgentCard—a JSON-formatted capability advertisement detailing the agent's specific name, input and output modalities, semantic descriptions, and execution skills—and subsequently exposes the agent via a dedicated, embedded Uvicorn web server.[1]

A2A facilitates this asynchronous task management through Server-Sent Events (SSE) and

robust task abstraction. When the Orchestrator assigns the Summarizer agent the task of synthesizing a multi-chapter review into a cohesive multimedia format, the Summarizer registers the operation as a distinct Task object, returning a unique task_id to the client.[1] The Orchestrator continuously monitors the execution stream via SSE, enabling the 5sosy frontend to display real-time, granular progress indicators to the student without maintaining a heavy, persistent HTTP connection.[1]

The Orchestration Layer and Semantic Routing

The central node of the 5sosy network is the Orchestrator Agent, which functions as the primary cognitive router. Efficient routing is critical for minimizing latency and optimizing API token expenditure. The orchestrator utilizes a semantic routing logic defined explicitly within its root system prompt.[1]

If a student's query involves a simple, single-paragraph lookup, a localized definition, or a brief paraphrase, the Orchestrator identifies the low cognitive complexity of the request and routes it immediately to the Select-and-Explain agent operating on a high-speed, lightweight model.[1] Conversely, if the query requires comparing two disparate textbook sections, executing multi-step mathematical reasoning, building a longitudinal study plan, or generating external media artifacts, the Orchestrator escalates the request, delegating the operation to the appropriate heavyweight reasoning agents.[1] Furthermore, if the query inherently references the student's historical performance, the Orchestrator autonomously triggers a sub-routine to fetch context from the Profiling agent before dispatching the primary task.[1]

Multi-LLM Comparative Analysis and Selection Strategy

Initial brainstorming for the 5sosy platform considered a multi-LLM architecture, hypothesizing that integrating Anthropic's Claude and OpenAI's GPT models alongside Google's Gemini could provide a diversified reasoning baseline.[1] However, an exhaustive architectural risk assessment and feasibility study strongly contraindicates this heterogeneous strategy, ultimately leading to the selection of a strictly Gemini-exclusive foundation.[1]

The Linguistic Imperative: Arabic VLM Performance

The 5sosy platform relies on the ingestion of highly complex, unstructured Arabic educational materials. Traditional Optical Character Recognition (OCR) pipelines struggle profoundly with the Arabic language due to its cursive script, right-to-left text flow, and the complex typographic features inherent in Ministry of Education textbooks.[8]

To evaluate model performance in this specific domain, the architecture relies on empirical data from the KITAB-Bench study—a comprehensive multi-domain benchmark for Arabic OCR and Document Understanding.[9] Evaluation metrics from this study indicate that modern Vision-Language Models (VLMs) significantly outperform legacy OCR engines (such as

Tesseract or EasyOCR) by an average of 60 percent in Character Error Rate (CER).[1] Crucially, models within the Gemini 2.0 and 2.5 families consistently achieve best-in-class results, reaching a 65 percent accuracy rate on the Markdown Recognition Score (MARS) for Arabic PDFs.[1] While Claude Opus and GPT-4o are highly competitive in general reasoning tasks, they do not demonstrate a statistically significant superiority in Arabic multimodal extraction capable of justifying the integration overhead.[1]

Hackathon Judging Optics and Technical Cohesion

Beyond raw performance metrics, the strategic context of the Google for Startups AI Agents Challenge must dictate architectural decisions. The official evaluation criteria weight Technical Implementation at 30 percent, with a strong, implicit emphasis on the cohesive utilization of the Google Cloud ecosystem.[1]

Integrating external, non-Google LLMs via third-party APIs introduces severe technical and optical risks. It bifurcates the billing infrastructure, complicating the utilization of the provided USD 37,500 in Google Cloud credits.[1] It introduces dual-vendor integration latencies and necessitates the maintenance of parallel SDKs within the 17-day development window. Most critically, an over-reliance on external inference engines risks a severe scoring penalty from the judging panel, as it detracts from demonstrating mastery over Google's native generative AI suite.[1]

Consequently, 5sosy establishes a hierarchical Gemini-only baseline. Gemini 2.5 Pro is deployed as the heavyweight engine for complex reasoning, multimodal document ingestion, and pedagogical analysis. Gemini 2.5 Flash operates as the rapid routing and localized orchestration engine. Finally, Gemini Flash Lite is utilized within the callback architecture to provide instantaneous, cost-effective security guardrails and prompt-injection filtering.[1]

Exhaustive Agent Topology

The 5sosy platform decentralizes cognitive load across a network of twelve highly specialized agents. Each agent is instantiated with specific toolsets, distinct ADK framework types, and tailored system instructions to ensure optimal execution within its bounded domain.[1]

The following table details the comprehensive topology of the multi-agent system:

Agent Designation	Underlying Model	ADK Framework Type	Associated Tools & Integrations	Primary Inputs	Output Artifacts
1. Orchestrato	Gemini 2.5	LlmAgent	A2A Clients	User query, session	Dispatched

r	Flash	(Root)		context	network tasks
2. Ingestion	Gemini 2.5 Pro	SequentialAgent	Cloud Storage, Vector Search	PDF URLs, raw documents	Structured Markdown chunks
3. Pedagogical Analysis	Gemini 2.5 Pro	LlmAgent	None (Pure Reasoning)	Markdown chunks, grade level	Bloom's taxonomy tags
4. Chat-with-Document	Gemini 2.5 Pro	LlmAgent w/ RAG	Vector Search, Firestore	User queries, document scope	Grounded, cited answers
5. Select-and-Explain	Gemini 2.5 Flash/Pro	LlmAgent	Vector Search	Text bounding boxes, mode	Contextual explanations
6. Summarizer	Gemini 2.5 Pro	ParallelAgent	Image-extraction tools	Chapter IDs, format requests	Video manifests, text summaries
7. Diagnostic Assessment	Gemini 2.5 Pro	LlmAgent	Vector Search, Firestore	Topic constraints, time budget	JSON quizzes, flashcards
8. Audio-Visual Synthesis	Gemini 2.5 Pro	SequentialAgent	ElevenLabs API, Remotion	Text summaries, duration reqs	Synthesized MP4 media

9. Study Planner	Gemini 2.5 Pro	LlmAgent	Notion MCP	Exam dates, hourly capacity	Generative Notion page trees
10. Profiling / Onboard	Gemini 2.5 Flash	SequentialAgent	Firestore	Diagnostic wizard answers	Persistent user profiles
11. Feedback / Insights	Gemini 2.5 Flash	LlmAgent	Firestore	Telemetry signals, ratings	Categorized insights
12. Loop / Batch Update	Gemini 2.5 Flash	LoopAgent	Pub/Sub, A2A Client	Monthly chronologies	Source-of-truth updates

Detailed Agent Operations and Workflows

The Ingestion and Topology Agent: This agent is responsible for the foundational structural asset compilation. It bypasses traditional OCR methodologies, instead utilizing the Gemini 2.5 Pro Multimodal API to parse raw educational PDFs directly.[1] It extracts tables, charts, and mathematical layouts, translating them into structured Markdown.[1] To preserve high fidelity, the agent isolates complex physics equations and scientific figures, treating them as discrete image artifacts mapped contextually to the text rather than attempting textual conversion.[1]

The Pedagogical Analysis Agent: Operating strictly on pure reasoning without external tool access, this agent functions as the primary instructional designer.[1] It ingests the raw parsed documents generated by the Ingestion Agent and maps out prerequisites, core conceptual nodes, and potential areas of student misconception. It appends standard educational metadata, such as Bloom's Taxonomy tags and difficulty indices, to every extracted text chunk, ensuring that subsequent operations maintain rigorous academic standards.[1]

The Chat-with-Document Agent: This agent utilizes a standard Retrieval-Augmented Generation (RAG) framework, connecting to Vertex AI Vector Search and Firestore. When a student asks a specific question regarding a textbook chapter, this agent retrieves the most semantically relevant text chunks. It operates under strict citation-required guardrails,

programmed to refuse an answer if the top-k retrieval score falls below an acceptable confidence threshold, thereby minimizing the risk of hallucinatory instruction.[1]

The Select-and-Explain Agent: Designed for frictionless, localized interactions, this agent processes UI-driven bounding box selections. When a user long-presses a specific paragraph or image within the web application, they can request a brief, detailed, or simplified explanation. The agent contextualizes the specific snippet within the broader chapter framework, returning an inline explanation tailored to the student's specific reading level as defined in their Firestore profile.[1]

The Document Summarizer Agent: Utilizing a ParallelAgent structure, this agent can synthesize massive amounts of text simultaneously by deploying multiple execution branches across different book chapters.[1] It can generate standard textual summaries interspersed with extracted photographs, or it can format the output into a structured manifest specifically designed for downstream video rendering.[1]

The Diagnostic Assessment Agent: This agent dynamically authors interactive, contextual exams, multiple-choice questions, and spaced-repetition flashcards.[1] It generates these assessments in a strict JSON schema, utilizing the Vector Search database to ensure every question maps directly back to the official Ministry of Education curriculum. Furthermore, it tracks the student's scoring trajectory over time, writing telemetry records into the Firestore database to isolate precise failure models.[1]

The Loop/Batch Update Agent: Operating as an autonomous chronological worker, this LoopAgent is triggered monthly via Google Cloud Scheduler and Pub/Sub architectures.[1] It independently navigates to the Ministry of Education's digital repository, calculates SHA-256 hashes against the currently ingested textbooks, and autonomously triggers the Ingestion Agent to update the Vector Search index if new curriculum revisions are detected.[1]

Data Ingestion, Vector Topology, and State Management

Transforming static PDF files into a dynamic, queryable vector space requires sophisticated infrastructure, particularly when dealing with an entire national curriculum spanning multiple grade levels and scientific disciplines.

Vector Search and Multilingual Embedding Strategy

Following the multimodal extraction process, the raw Markdown text is systematically chunked to preserve semantic meaning. The chunking algorithm executes a hierarchical split—dividing the text by structural markers derived from the table of contents (chapter, section, subsection), and subsequently splitting by paragraph utilizing a 200-token soft minimum and a 256-token overlap window.[1] Each resulting chunk is enriched with highly specific metadata, including the book ID, grade level, subject, chapter ID, page number, language flag, and boolean indicators

for the presence of mathematical formulas or figures.[1]

These chunks are embedded using the gemini-embedding-001 model. This model is explicitly selected over open-source alternatives (such as pgvector or local HuggingFace embeddings) because it natively supports 3072-dimensional multilingual vectorization with robust Arabic support.[1] During the embedding process, the system leverages task-specific parameters: RETRIEVAL_DOCUMENT is utilized during the initial ingestion of the textbooks, RETRIEVAL_QUERY is applied to incoming user chats to optimize similarity matching, and CLASSIFICATION is used to categorize telemetry data within the diagnostic loop.[1] The resulting vectors are indexed within Vertex AI Vector Search utilizing cosine distance metrics and a Hierarchical Navigable Small World (HNSW) algorithm to facilitate approximate, low-latency nearest-neighbor retrieval at scale.[1]

Database Design: Session Persistence and Profiling

Because 5sosy functions as a longitudinal educational mentor rather than an episodic search engine, ephemeral memory architectures are insufficient. The platform requires robust, serverless long-term state tracking.[11] The ADK natively supports session management through the DatabaseSessionService, and 5sosy implements this utilizing Google Cloud Firestore in Native mode.[12]

Firestore operates as a highly scalable NoSQL document database, allowing the backend to map Python dictionaries representing ADK Session objects directly into document schemas.[13] The database architecture consists of several primary collections:

1. **Users:** Stores authenticated profiles, integrating seamlessly with Firebase Auth (which manages Google Sign-In and standard email credentials). This collection houses diagnostic information gathered during onboarding, including the student's grade, academic stream, language preference, and identified weak topics.[1] It also securely stores encrypted OAuth refresh tokens for third-party integrations.[1]
2. **Sessions:** Manages the active ADK conversation state, appending user queries and agent responses as sub-collection events, ensuring that conversational context is preserved even if the underlying Cloud Run container restarts.[13]
3. **Telemetry and Insights:** A persistent ledger of the student's historical performance on quizzes and flashcards, powering the spaced-repetition algorithms and the weekly insights dashboard.[1]

To support the storage of raw PDFs and generated media, the system relies on Google Cloud Storage. Three dedicated buckets are instantiated: 5sosy-textbooks-raw for the original Ministry of Education files, 5sosy-figures for the canonical image crops of extracted diagrams, and 5sosy-artifacts for the final generated MP4 videos and MP3 audio files.[1] A lifecycle policy automatically transitions artifacts older than 30 days into Coldline storage to minimize operational costs.[1]

Advanced Tooling: Notion MCP and API Registry Integration

To extend the cognitive capabilities of the agent network beyond basic chat interfaces, the architecture relies heavily on the Model Context Protocol (MCP). MCP standardizes how AI applications interact with external data sources and enterprise tools, eliminating the need to construct brittle, bespoke API wrappers for every integration.[15]

Dynamic Study Planning via Notion MCP

A critical differentiator for the 5sosy platform is the Study Planner Agent, which is capable of dynamically drafting long-term study calendars based on a student's impending exam dates and available weekly study hours.[1] To provide maximum utility to the student and offer passive visibility to parents, these study plans are exported directly out of the 5sosy ecosystem and into Notion.

This integration leverages the officially hosted Notion MCP server (https://mcp.notion.com/mcp) via a Streamable HTTP transport layer.[1] This remote server grants the Study Planner Agent access to highly optimized, AI-specific tools, including notion-search, notion-create-pages, and notion-update-pages.[1] The agent utilizes these tools to programmatically construct a nested page tree within the user's workspace, detailing daily reading assignments and linking back to specific 5sosy review sessions.[1]

A significant architectural challenge in this implementation involves authentication. The official Notion MCP server mandates user-based OAuth and does not support persistent, hardcoded bearer tokens for multi-tenant applications.[1] Consequently, 5sosy implements a rigorous OAuth 2.0 Authorization Code flow utilizing Proof Key for Code Exchange (PKCE).[7] Employing Dynamic Client Registration (RFC 7591), the ADK client automatically registers itself with Notion during the initial user setup, triggering a standard browser-based consent flow.[18]

Because the 5sosy backend operates within stateless Cloud Run containers, it cannot maintain authorization states in volatile memory. Upon successful completion of the OAuth callback, the backend exchanges the temporary authorization code for persistent access and refresh tokens. These tokens are encrypted and written directly into the user's Firestore profile document.[1] During subsequent planning operations, the Study Planner Agent retrieves the refresh token, silently re-authenticating the MCP HTTP streamable connection without requiring repeated user intervention.[7]

Google Cloud API Registry Integration

In addition to integrating third-party MCP servers, 5sosy utilizes the Google Cloud API Registry to connect internal GCP services directly to the agent layer.[1] By instantiating the ADK's ApiRegistry class, the development environment retrieves specialized McpToolset objects that natively wrap services like BigQuery.[1] Application Default Credentials (ADC) handle the

authentication seamlessly, ensuring that the agents possess the necessary roles/mcp.toolUser Identity and Access Management (IAM) bindings to execute administrative tool calls, facilitating deep telemetry analytics without exposing raw infrastructure secrets.[1]

Programmatic Media Synthesis and Multimodal Pipelines

To accommodate varying educational learning styles and to simulate the interactive nature of high-tier private tutoring, 5sosy incorporates an autonomous media generation pipeline. The Audio-Visual Synthesis Agent transforms textual summaries into high-fidelity, narrated video explainers, dynamically rendered in one, three, or five-minute variants depending on the user's request.[1]

Video Assembly via Remotion on Cloud Run

The system utilizes Remotion, a framework that allows programmatic video definition using standard React components.[19] Rather than relying on expensive, unpredictable, black-box generative AI video models that frequently hallucinate details, Remotion ensures deterministic, curriculum-accurate rendering. It operates by manipulating HTML DOM elements and stitching the resulting frame sequences utilizing FFmpeg.[19]

Deploying a headless video rendering engine introduces severe computational challenges. Standard Serverless functions often encounter hard timeouts during intensive video encoding processes.[19] To circumvent this, the 5sosy video architecture wraps the Remotion renderer within a custom Docker container, configured with Puppeteer and necessary Chromium dependencies, and deploys it as an asynchronous Google Cloud Run Job.[1]

However, Cloud Run compute instances lack dedicated Graphics Processing Units (GPUs).[21] Complex graphical effects, off-thread video processing, and high-resolution rendering can create severe CPU bottlenecks.[21] To mitigate this architectural limitation, the Remotion templates rely strictly on static assets, precomputed images (extracted directly from the Ministry of Education PDFs via the Ingestion Agent), and CSS-level animations rather than heavy WebGL effects.[21] Furthermore, rendering speed is optimized by carefully tuning the --concurrency flag during deployment and allocating a high-vCPU configuration specifically for the generation job, reducing the render time for a standard one-minute explainer to approximately 30 seconds.[1]

High-Fidelity Arabic Speech-to-Text and Text-to-Speech

Audio narration for the Remotion pipeline is driven by ElevenLabs. While an MCP integration exists for this service, 5sosy opts to utilize direct HTTP API calls to the ElevenLabs Multilingual v2 model.[1] Bypassing the MCP layer for this specific task reduces protocol overhead, allowing for rapid batch synthesis and achieving generation latencies of approximately 75 milliseconds.[1]

The Multilingual v2 model natively supports Arabic and excels at maintaining consistent emotional resonance, natural pacing, and precise articulation.[22] Intensive testing indicates that the model's default Arabic vocalizations lean toward Saudi or UAE dialects rather than localized Egyptian Arabic.[1] However, careful prompt engineering utilizing explicit dialectal instructions, coupled with the selection of a warm, approachable speaker profile, yields production-quality educational voiceovers.[24] During the assembly process, the Audio-Visual Synthesis Agent generates the MP3 narration via ElevenLabs, calculates the precise audio duration, and subsequently instructs the Remotion container to adjust the video's total durationInFrames to match the audio track perfectly, ensuring flawless synchronization.[1]

Development Environment: Local Integration and Antigravity IDE

The transition from the prototype phase—initially housed in the local directory C:\Users\hesh1\Desktop\my-agent—to the production-ready architecture at C:\Users\hesh1\Desktop\5sosy necessitates a robust development environment.[1] To maximize coding velocity and maintain strict alignment with the Google Cloud ecosystem, the development cycle integrates Google's newly released Antigravity IDE.[1]

Antigravity operates as an agent-first Integrated Development Environment, bringing autonomous AI coding capabilities directly into the local workspace.[26] Recognizing that an AI coding agent's utility is strictly limited by its contextual understanding of the developer's data, the IDE features a native MCP store.[28] During the scaffolding phase of the 5sosy project, the Antigravity IDE is connected directly to the Google Cloud project via Identity and Access Management (IAM) credentials.[28] This secure MCP connection allows the local coding agent to autonomously query the production Firestore database, inspect the Vertex Vector Search indices, and read logs from Cloud Run without exposing raw developer secrets within the chat window, dramatically accelerating the debugging and integration of the complex A2A agent network.[28]

Security Architecture, Guardrails, and Execution Sandboxing

Allowing an autonomous network of twelve agents to query vector databases, write data to external Notion workspaces, and execute generated code requires rigorous, multi-layered security protocols. The architecture must actively prevent data exfiltration, indirect prompt injections, and hallucinatory actions.[1]

Execution Interception via ADK Callbacks

The Agent Development Kit features a highly granular event lifecycle managed through programmable Callbacks. These functions allow developers to intercept and inspect agent behavior at critical junctures—specifically before executing an external tool

(before_tool_callback) and before interacting with the LLM (before_model_callback).[29]

Within the 5sosy architecture, the before_tool_callback serves as the primary security gateway. For example, when the Study Planner Agent decides to invoke the notion-create-pages MCP tool, the callback intercepts the request mid-flight. The function verifies the requested payload parameters against the user's explicit instructions, checks the current state of the agent, and dynamically injects the correct OAuth bearer token fetched securely from Firestore.[1] If the agent hallucinates and attempts to modify a Notion database outside the defined scope of the study plan, the callback explicitly returns an error state, preventing the tool execution entirely.[1]

Furthermore, a comprehensive PII Redaction Plugin operates globally within this callback layer. This plugin ensures that any sensitive student telemetry or personally identifiable information is meticulously scrubbed before being passed to any third-party external services, such as ElevenLabs or Notion.[1]

The "Gemini as a Judge" Protocol

Adversarial inputs pose a continuous threat to the stability of an educational platform. Students may attempt direct prompt injections (jailbreaks) to bypass quiz logic and extract the correct answers directly from the model. Similarly, indirect prompt injections—malicious instructions hidden within poorly parsed or compromised external document texts—can compromise the agent's behavior.[1]

To neutralize these attack vectors dynamically, 5sosy employs a high-speed "Gemini as a Judge" plugin.[1] Operating within the ADK callback architecture, this plugin leverages the highly efficient and cost-effective Gemini Flash Lite model as an active, external safety filter. Every incoming user input, tool output, and final generated response is routed through Gemini Flash Lite for a rapid security evaluation. If the judge model detects a jailbreak attempt, toxic language, or an ungrounded claim (a hallucination), the primary execution thread is instantly halted. The system then gracefully degrades, returning a safe, predetermined fallback response, ensuring that the student is never exposed to inappropriate or factually inaccurate material.[1]

Sandboxed Code Execution and Output Escaping

Because 5sosy operates dynamically, it occasionally requires the generation and execution of code—for instance, when a student requests a step-by-step mathematical proof or a complex data visualization. All model-generated code is strictly confined and executed within the Vertex Code Interpreter Extension.[1] This hermetic sandbox prevents any unauthorized network connections or API access that could lead to host-level compromise or cross-user data leakage.[1]

Additionally, to prevent malicious scripts from executing within the browser context, the 5sosy web application strictly escapes all model-generated content. Unescaped UI inputs could theoretically allow an adversarial prompt to trick the model into rendering harmful HTML or

JavaScript tags, leading to severe vulnerabilities such as session cookie theft. Proper output escaping guarantees that the Next.js frontend renders all LLM outputs strictly as safe, sanitized plain text.[1]

Product Blueprint and User Experience (UX) Architecture

The immense technical depth of the multi-agent system must be abstracted entirely from the end-user. The primary persona comprises Egyptian secondary students and their parents. This demographic frequently relies on low-end Android devices (such as the Samsung A-series) and unstable Wi-Fi or 4G connections to access the internet.[1]

Information Architecture and Core User Journeys

The product's information architecture prioritizes a low-friction onboarding wizard designed to capture essential diagnostic information in under four minutes.[1] Students authenticate seamlessly via Firebase Auth (Google Sign-In), select their educational stage, specify their curriculum stream (e.g., Science, Mathematics, or Literature), choose their language preference, and indicate their weekly study hour capacity.[1] The wizard concludes with a brief, ten-question diagnostic assessment to immediately identify academic weak points.[1]

Once onboarded, the user is directed to the primary interface: the Library. This centralized hub is populated continuously by the autonomous Loop Agent executing its monthly MoE synchronization routines.[1] The core study loop involves the student opening a specific textbook chapter. The interface features a responsive 60/40 split: the primary reader column displays the structured text and visual figures extracted from the original PDF, while a persistent right-hand rail houses the tabbed agent interactions (Chat, Quiz, Flashcards, Summary, Video).[1]

Through this interface, students execute declarative intents using natural language. A student highlighting a confusing paragraph triggers a floating action menu, summoning the Select-and-Explain agent. The agent processes the specific textual bounding box and returns an inline, hyper-localized explanation in simplified Arabic.[1] If the student's telemetry indicates repeated failures on a specific concept over time, the Insights agent proactively surfaces recommendations, prompting the user to take a localized adaptive quiz or generate a one-minute Remotion review video.[1]

UI/UX Engineering and Low-Latency Delivery

The 5sosy frontend is engineered using Next.js 15 (App Router) combined with Tailwind CSS and the lightweight shadcn/ui component library.[1] By aggressively leveraging React Server Components (RSC) and Server Actions, the framework minimizes the client-side API surface area and dramatically reduces the initial JavaScript payload size—a critical engineering requirement for targeting low-end mobile devices common in the Egyptian market.[1]

To support the Arabic language natively and flawlessly, the system utilizes global logical CSS

properties and dynamic dir="rtl" contextual switching. The UI avoids static directional icons, opting instead for logical chevron-start and chevron-end variants that automatically flip based on the user's language selection.[1] The platform also correctly implements Eastern Arabic numerals (٩-٠) within the Arabic UI, while maintaining standard Western numerals exclusively inside mathematical equations, utilizing unicode-bidi: plaintext to prevent rendering reversals in mixed-language paragraphs.[1]

Typography utilizes the highly legible IBM Plex Sans Arabic font, with fallback layers to Tajawal and Cairo for headings. The design system enforces strict WCAG AA color contrast ratios, utilizing a deep teal and warm gold palette, to compensate for low-DPI mobile screens.[1] Furthermore, the architecture incorporates progressive enhancement via Service Workers, allowing the application to silently cache recently viewed chapters and generated explainer videos, providing high offline tolerance during the inevitable regional network disruptions.[1]

Market Feasibility and Competitive Landscape

The integration of advanced AI architectures into the Egyptian educational sector requires an objective analysis of the existing socio-economic landscape and the competitive environment. The proliferation of the *Deroos Khoseya* system places immense, often unsustainable financial strain on middle and lower-class families.[1] With the average monthly salary of an Egyptian teacher resting near EGP 7,750 (approximately USD 157), educators frequently rely on private tutoring to subsidize their income, pricing their services at a significant premium.[1]

The Competitive Environment

The current EdTech ecosystem in Egypt and the broader MENA region is heavily populated by platforms digitizing the traditional synchronous tutoring model, rather than fundamentally reimagining the pedagogical approach through AI.

The following table outlines the primary competitors within the regional market:

Competitor	**Core Offering**	**Pricing Model**	**5sosy Differentiator**
Nagwa Classes	Live human tutors, curriculum-aligned content	Wallet-based per-session	Autonomous 24/7 AI availability; multimodal generation
Orcas / Baims	Tutoring marketplace, K-12 focus, pre-recorded	Subscription / Session	Dynamic, personalized interaction; non-static

	video		media
Khanmigo	GPT-4 powered Socratic AI tutor	$4/month subscription	Egyptian MoE curriculum alignment; native Arabic
Praktika	AI English-only tutor with avatar	Monthly subscription	Focus on core scientific/mathematical curriculum
Almentor	Arabic MOOCs, professional development	Subscription	Focus exclusively on K-12 standardized testing

Market incumbents like Nagwa Classes operate on a wallet-based, per-session pricing structure, relying entirely on scheduled human interactions.[1] Similarly, platforms like Orcas—recently acquired by the Kuwaiti ed-tech firm Baims in an $11 million strategic exit—focus on connecting students to human tutors or providing pre-recorded, static video courses.[3][4]

While global platforms such as Khan Academy's Khanmigo offer advanced Socratic interactions driven by GPT-4, they lack regional curriculum alignment and Arabic-first multimodal support, rendering them highly ineffective for rigorous, culturally specific *Thanaweya Amma* exam preparation.[1]

The 5sosy Competitive Moat

The 5sosy platform establishes a highly defensible market moat through three intersecting strategic vectors:

1. **Deterministic Curriculum Alignment:** Unlike general-purpose conversational chatbots, 5sosy utilizes the official Egyptian Ministry of Education PDFs as its foundational ground truth. This ensures that every generated quiz, study plan, and video is explicitly aligned with national testing standards, maintaining pedagogical integrity.[1]
2. **Autonomous Asynchronous Economics:** By replacing expensive synchronous human labor with an autonomous, highly optimized agentic system, 5sosy dramatically undercuts the pricing structure of legacy platforms. AI inference costs operate at

fractions of a cent per query, allowing for an aggressively priced premium subscription tier (estimated at approximately USD 3.99 per month) that democratizes access for lower-income demographics while maintaining high enterprise margins.[1]

3. **Multimodal Interoperability:** By supporting localized audio synthesis, dynamic programmatic video generation, and the strict preservation of complex visual diagrams, the platform accommodates multiple learning modalities simultaneously—a feature set technically unachievable in legacy recorded-video platforms.

From a legal feasibility standpoint, the platform operates within the bounds of Egyptian copyright law (Law 82/2002), which provides clear fair-dealing exemptions for educational, non-commercial, and transformative use.[1] The ingestion, chunking, and Retrieval-Augmented Generation processes constitute a fundamentally transformative application of the publicly distributed Ministry of Education materials.[1]

Comprehensive Strategy and Project Planning

To successfully submit a production-grade prototype to the Google for Startups AI Agents Challenge (Track 1) by the June 5, 2026 deadline, the development process must adhere to strict scope control and operational discipline.[1] The official evaluation criteria weight Technical Implementation at 30 percent, Business Case at 30 percent, Innovation and Creativity at 20 percent, and the Demo/Presentation at 20 percent.[1]

MVP Scope Definition and Stretch Goals

The most severe operational risk to the solo-developer project is "scope creep." Attempting to ingest and validate the entirety of the 12-year Egyptian public curriculum prior to the deadline will drastically diffuse the impact of the final demonstration.[1] Therefore, the ingestion pipeline will focus exhaustively on a single, high-complexity subject: *Thanaweya Amma* Year 3 Physics.[1] Demonstrating extreme algorithmic depth—where agents can accurately discuss complex mechanics, generate targeted physics quizzes, and render accurate diagrams—provides a much stronger signal to the judges than superficial, error-prone coverage of a dozen disparate textbooks.[1]

The Minimum Viable Product (MVP) isolates seven critical agents necessary for the core demo: The Orchestrator, the Ingestion Agent, the Chat-with-Document Agent, the Select-and-Explain Agent, the Summarizer, the Quiz Agent, and Audio-Visual synthesis restricted strictly to 1-minute formats.[1] Stretch goals, including the complex Insights telemetry dashboard, the computationally expensive 3- and 5-minute video generation variants, and the fully autonomous monthly Loop cron jobs, will be stubbed, hardcoded, or simulated for the submission if the primary timeline begins to slip.[1]

17-Day Execution Timeline

The following table outlines the rigorous, day-by-day development chronology spanning from

project inception (May 19) to final submission (June 5):

Date	Development Focus	Key Deliverable
May 19	Scaffold monorepo at C:\Users\hesh1\Desktop\5sosy; confirm Devpost rules; authenticate GCP project APIs.	Initialized repo, verified Egypt eligibility.
May 20	Ingest History_Sec3.pdf sample; deploy Gemini 2.5 Pro multimodal extraction pipeline; index to Vector Search.	First chapter searchable via embeddings.
May 21	Develop Chat-with-Document ADK agent; construct minimal Next.js UI (chapter viewer + chat panel).	Arabic RAG operational with citations.
May 22	Build Onboarding wizard; integrate Firebase Auth; establish Firestore native user profiles.	End-to-end user signup and diagnostic flow.
May 23	Deploy Select-and-Explain agent; implement frontend DOM text-selection handlers.	Contextual UI bounding-box explanations.
May 24	Construct Quiz agent utilizing vector search	Generative 10-question MCQ

	context; build Next.js quiz player UI.	assessments.
May 25	Implement ParallelAgent Summarizer (text + embedded photo extraction logic).	End-to-end chapter summarization.
May 26	Integrate ElevenLabs Multilingual v2 API; generate first synthesized Arabic voiceover.	Audio playback integrated into summary UI.
May 27	Deploy Remotion video service as a Cloud Run Job; stitch frames and audio into MP4.	First programmatic video artifact in GCS.
May 28	Implement Notion MCP server connection; engineer OAuth 2.1 flow and token persistence.	Working "Send to Notion" study plan button.
May 29	Develop spaced-repetition flashcards; implement basic Insights telemetry aggregation (stubbed if delayed).	Flashcard player; rudimentary dashboard.
May 30	Deploy Loop Agent via Cloud Scheduler; ingest three additional chapters for demo	Multi-chapter library populated.

	breadth.	
May 31	Execute UI/UX polish; verify dir="rtl" fidelity; finalize IBM Plex Sans Arabic typography.	Production-grade aesthetic presentation.
June 1	Conduct end-to-end bug fixing; run 100-question Arabic eval harness.	Latency and hallucination metrics logged.
June 2	Draft README, Mermaid architecture diagrams, and comprehensive Devpost written submission.	Textual submission artifacts finalized.
June 3	Record v1 of the 3-minute demonstration video; A/B test voiceover pacing and clarity.	Rough cut of presentation video.
June 4	Finalize video edit; deploy production Cloud Run environment with custom domain; execute load tests.	Production URL live and stable.
June 5	Submit to Devpost prior to 12:00 PT to buffer against 17:00 PT deadline.	Confirmed Track 1 Submission.

Deployment Operations and Risk Mitigation

The entire backend and frontend infrastructure will be deployed onto Google Cloud Run. The

Next.js application, the primary ADK Orchestrator API server, and the remote A2A sub-agents will operate as distinct, horizontally scalable container services.[1]

Deploying the Python ADK environment requires a highly structured repository layout. The agent code must be housed in modular directories utilizing __init__.py files, and the adk deploy cloud_run Command-Line Interface (CLI) will automate the Google Cloud Build process.[1] Authentication to these services is strictly governed; unauthenticated public access will be restricted at the container level, requiring caller clients to supply Google Cloud identity tokens via HTTP Authorization headers to prevent unauthorized execution.[1]

By unifying the infrastructure under a single Google Cloud billing account—expressly avoiding third-party hosting platforms like Vercel—the submission aligns perfectly with the sponsor's technical optics and judging rubrics.[1]

A comprehensive risk register governs the final execution phase. If the Ministry of Education website alters its URL structure mid-build, the system will fall back to hardcoded, locally stored sample PDFs to ensure the Ingestion Agent remains operational for the demo.[1] The required RFC 7591 dynamic OAuth flow for the Notion MCP integration requires intense state management; if the integration consumes more than 48 hours of development time, the feature will be downgraded to target a hardcoded, developer-owned Notion workspace to ensure the core demo remains fluid.[1] Finally, to mitigate the latency of container cold starts during the final video recording, the Cloud Run services will be pre-warmed using the min-instances=1 configuration.[1]

Strategic Conclusions

The 5sosy architecture represents a profound paradigm shift in scalable educational technology for the MENA region. By transitioning away from standard, prompt-driven LLM wrappers toward a sophisticated, decentralized multi-agent orchestration model governed by the A2A protocol and the Google ADK, the platform achieves the reasoning depth required to accurately simulate an elite human tutor.

The integration of the Model Context Protocol seamlessly connects this cognitive layer to tangible, external user outcomes—such as dynamically generated Notion study schedules—while programmatic media synthesis engines address distinct, multimodal learning preferences through programmatic video and high-fidelity Arabic audio generation. Anchored entirely within the Google Cloud ecosystem, fortified by extensive Firestore session persistence, and secured by multi-layered ADK execution callbacks, 5sosy is precisely engineered to fulfill the stringent requirements of the Google for Startups AI Agents Challenge.

Beyond the immediate constraints of the hackathon, the underlying technical architecture provides a highly viable, aggressively scalable business model poised to disrupt a billion-dollar legacy market, fundamentally democratizing access to high-quality, curriculum-aligned education for millions of Egyptian students.

Works cited

1. 5sosy.pdf
2. GitHub - google/adk-python: An open-source, code-first Python toolkit for building, evaluating, and deploying sophisticated AI agents with flexibility and control., accessed May 19, 2026, https://github.com/google/adk-python
3. A2A Agent Patterns with the Agent Development Kit (ADK) | by xbill | Google Cloud, accessed May 19, 2026, https://medium.com/google-cloud/a2a-agent-patterns-with-the-agent-development-kit-adk-aee3d61c52cf
4. Agent2Agent (A2A) is an open protocol enabling communication and interoperability between opaque agentic applications. · GitHub, accessed May 19, 2026, https://github.com/a2aproject/A2A
5. Getting Started with MCP, ADK and A2A - Google Codelabs, accessed May 19, 2026, https://codelabs.developers.google.com/codelabs/currency-agent
6. Getting Started with Agent2Agent (A2A) Protocol: A Purchasing Concierge and Remote Seller Agent Interactions on Cloud Run and Agent Engine | Google Codelabs, accessed May 19, 2026, https://codelabs.developers.google.com/intro-a2a-purchasing-concierge
7. Integrating your own MCP client - Notion Docs, accessed May 19, 2026, https://developers.notion.com/guides/mcp/build-mcp-client
8. KITAB-Bench: A Comprehensive Multi-Domain Benchmark for Arabic OCR and Document Understanding - ACL Anthology, accessed May 19, 2026, https://aclanthology.org/2025.findings-acl.1135.pdf
9. [2502.14949] KITAB-Bench: A Comprehensive Multi-Domain Benchmark for Arabic OCR and Document Understanding - arXiv, accessed May 19, 2026, https://arxiv.org/abs/2502.14949
10. mbzuai-oryx/KITAB-Bench: [ACL 2025] A Comprehensive Multi-Domain Benchmark for Arabic OCR and Document Understanding - GitHub, accessed May 19, 2026, https://github.com/mbzuai-oryx/KITAB-Bench
11. Feature Request: Native FirestoreSessionService for production-ready persistent sessions · Issue #3776 · google/adk-python - GitHub, accessed May 19, 2026, https://github.com/google/adk-python/issues/3776
12. Session State Management using Firestore - Agent Development Kit (ADK), accessed May 19, 2026, https://adk.dev/integrations/firestore-session-service/
13. Building a Firestore Session Service with ADK's Service Registry, accessed May 19, 2026, https://engineering.szns.solutions/building-a-firestore-session-service-with-adks-service-registry/
14. Extending Google ADK: Building a Custom Session Service with Firestore - Medium, accessed May 19, 2026, https://medium.com/google-cloud/extending-google-adk-building-a-custom-session-service-with-firestore-0fc4b74354bf
15. Building a Financial AI Agent with Google ADK, A2A and MCP - Medium, accessed May 19, 2026,

https://medium.com/google-cloud/building-a-financial-ai-agent-with-google-adk-a2a-and-mcp-084f5937f1e8
16. makenotion/notion-mcp-server - GitHub, accessed May 19, 2026, https://github.com/makenotion/notion-mcp-server
17. Notion MCP Server | Panther Docs, accessed May 19, 2026, https://docs.panther.com/ai/mcp-integrations/notion-mcp-server
18. Notion MCP Example | OAuth Callback - Kriasoft, accessed May 19, 2026, https://kriasoft.com/oauth-callback/examples/notion
19. I built a video rendering engine using React, Remotion, and Cloud Run. Here is how it works. : r/reactjs - Reddit, accessed May 19, 2026, https://www.reddit.com/r/reactjs/comments/1qdkws5/i_built_a_video_rendering_engine_using_react/
20. Dockerizing a Remotion app | Remotion | Make videos programmatically, accessed May 19, 2026, https://www.remotion.dev/docs/docker
21. Performance Tips | Remotion | Make videos programmatically, accessed May 19, 2026, https://www.remotion.dev/docs/performance
22. Models | ElevenLabs Documentation, accessed May 19, 2026, https://elevenlabs.io/docs/overview/models
23. ElevenLabs Comes Out of Beta and Releases Eleven Multilingual v2 - a Foundational AI Speech Model for Nearly 30 Languages, accessed May 19, 2026, https://elevenlabs.io/blog/elevenlabs-comes-out-of-beta-and-releases-eleven-multilingual-v2-a-foundational-ai-speech-model-for-nearly-30-languages
24. Should I use Multilingual V2 or V3 Alpha for an Arabic Youtube Automation Brand? : r/ElevenLabs - Reddit, accessed May 19, 2026, https://www.reddit.com/r/ElevenLabs/comments/1ls9rr8/should_i_use_multilingual_v2_or_v3_alpha_for_an/
25. Multilingual AI Voices | ElevenLabs Voice Library, accessed May 19, 2026, https://elevenlabs.io/voice-library/multilingual
26. Firebase Studio Migration - Google Antigravity Documentation, accessed May 19, 2026, https://antigravity.google/docs/firebase-studio-migration
27. I tried Google's new Antigravity IDE so you don't have to (vs Cursor/Windsurf) - Reddit, accessed May 19, 2026, https://www.reddit.com/r/ChatGPTCoding/comments/1p35bdl/i_tried_googles_new_antigravity_ide_so_you_dont/
28. Connect Google Antigravity IDE to Google's Data Cloud services | Google Cloud Blog, accessed May 19, 2026, https://cloud.google.com/blog/products/data-analytics/connect-google-antigravity-ide-to-googles-data-cloud-services
29. 18. Google ADK - Callbacks - Arjun Prabhulal, accessed May 19, 2026, https://arjunprabhulal.com/adk-callbacks/
30. Callbacks: Observe, Customize, and Control Agent Behavior - Agent Development Kit (ADK), accessed May 19, 2026, https://adk.dev/callbacks/
31. adk-python/contributing/samples/plugin_basic/README.md at main - GitHub, accessed May 19, 2026, https://github.com/google/adk-python/blob/main/contributing/samples/plugin_ba

sic/README.md
32. Nagwa Classes, accessed May 19, 2026, https://www.nagwa.com/
33. Nagwa Pricing 2026: Cost and Pricing Plans - eLearning Industry, accessed May 19, 2026, https://elearningindustry.com/directory/elearning-software/nagwa/pricing
34. About Us - Orcas Tutoring - Your Partner in Personalized Learning, accessed May 19, 2026, https://www.orcas.io/about-us
35. Egyptian online tutoring startup Orcas acquired by Kuwaiti ed-tech firm Baims, accessed May 19, 2026, https://disruptafrica.com/2024/01/09/egyptian-online-tutoring-startup-orcas-acquired-by-kuwaiti-ed-tech-firm-baims/
36. Kuwaiti edtech Baim acquires Egypt's Orcas - Wamda, accessed May 19, 2026, https://www.wamda.com/2024/01/kuwaiti-edtech-baim-acquires-egypts-orcas
37. Agent Development Kit: Making it easy to build multi-agent applications, accessed May 19, 2026, https://developers.googleblog.com/en/agent-development-kit-easy-to-build-multi-agent-applications/
38. The Surprisingly Simple Way to Create an A2A Agent with ADK, Deploy on Cloud Run, and Register with Gemini Enterprise - Medium, accessed May 19, 2026, https://medium.com/google-cloud/surprisingly-simple-a2a-agents-with-adk-using-to-a2a-deploy-to-cloud-run-and-gemini-enterprise-e815bdef4a32